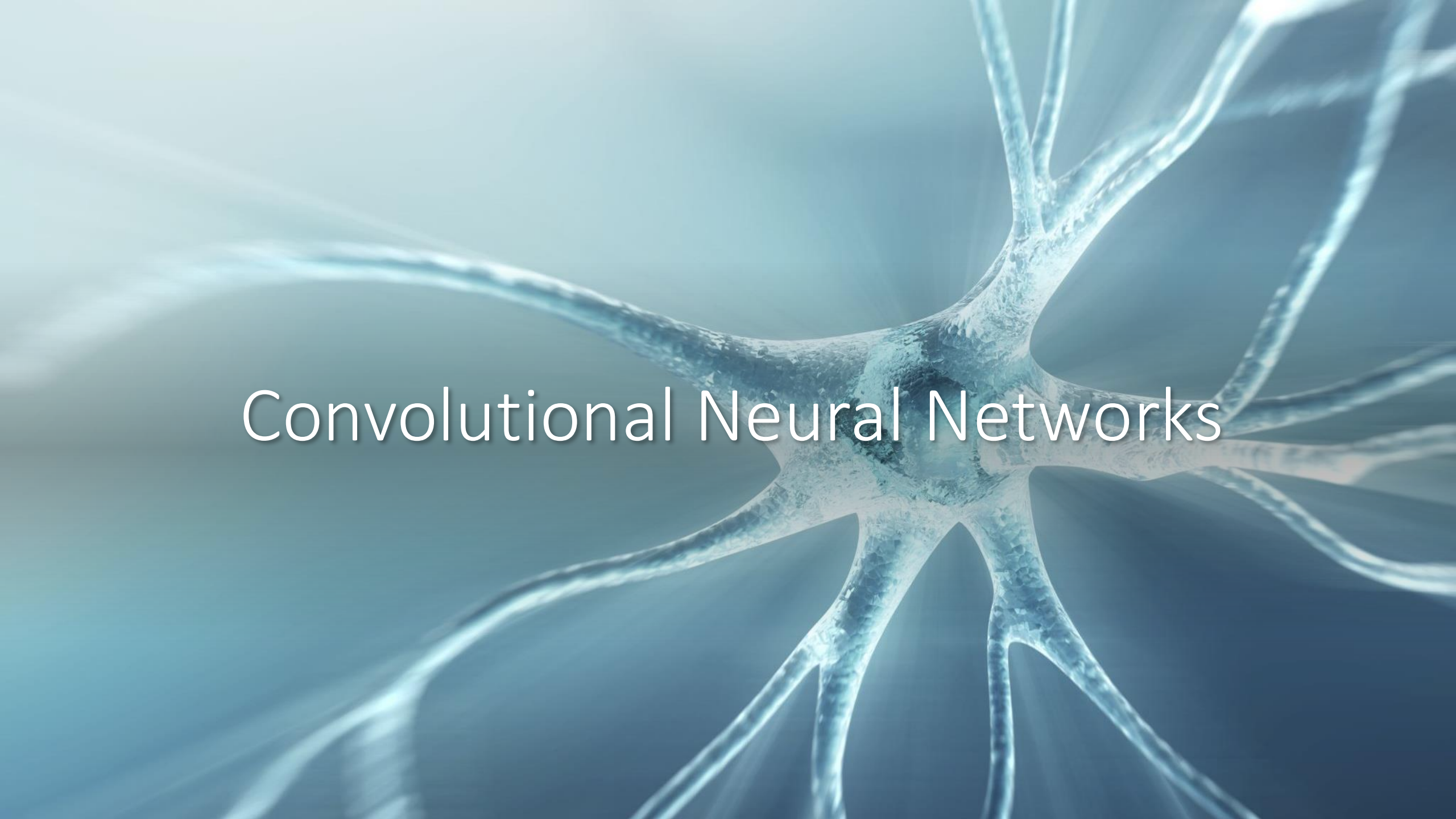


Convolutional Neural Networks (CNNs)

Priyanka Bagade, IITK

Lecture 6



Convolutional Neural Networks

Outline



IMAGE CLASSIFICATION
WITH MLP



UNDERSTANDING
CNNs



COLOR IMAGES

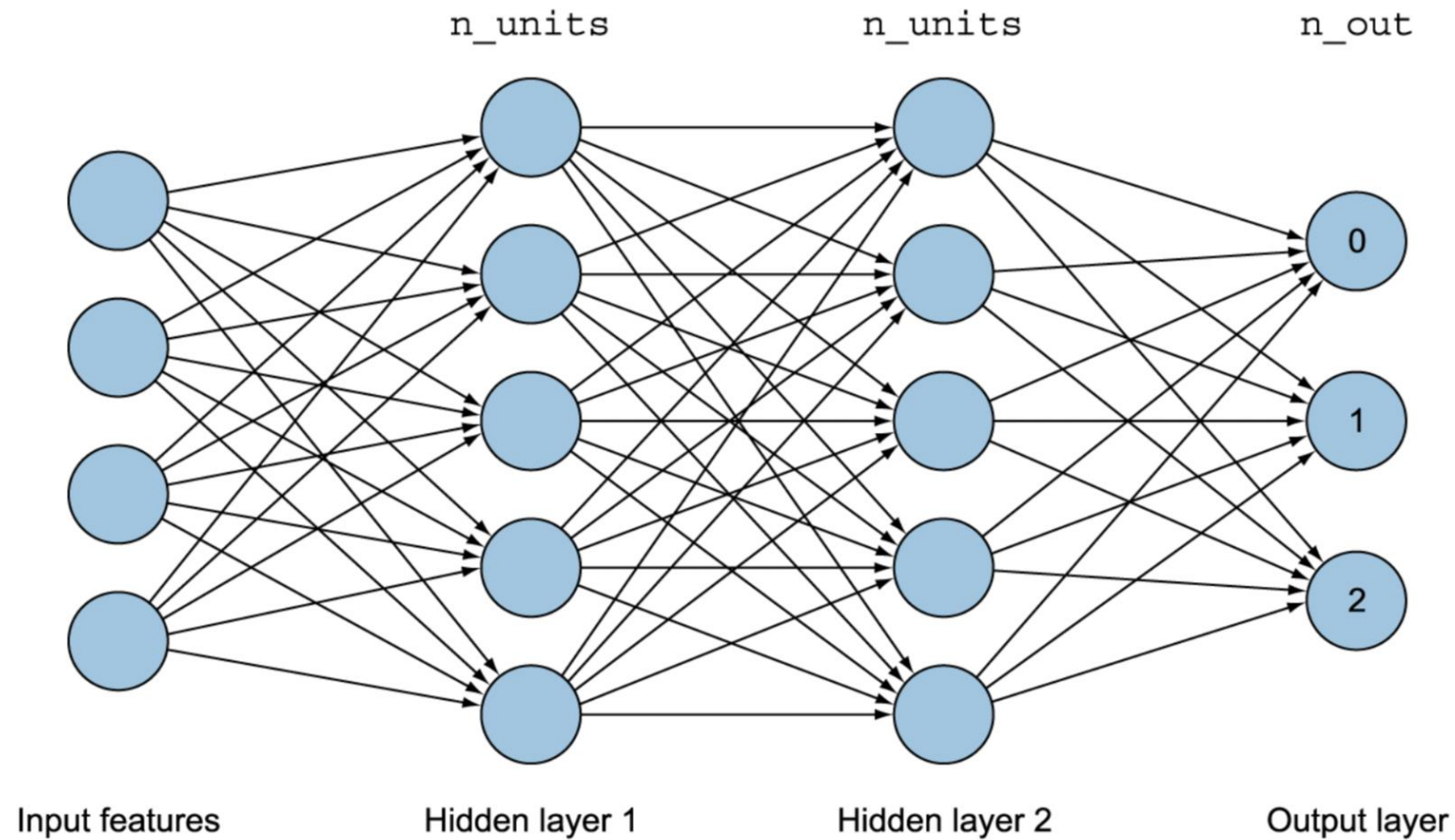


IMAGE CLASSIFICATION
PROJECT

MLPs vs CNNs

- Architecture – both contain stacked layers of neurons
- Weights and biases – both initialize weights and biases at random values and their values are learned by the network
 - Weights
 - MLP – vector
 - CNN – convolutional filters/kernels
- Hyperparameters – both have error function, activation function and optimizer. CNNs have additional hyperparameters apart from these.
- Training – both follow the same network learning process.
 - Feedforward->
 - calculate cost function->
 - optimize parameters using gradient descent ->
 - backpropagation->
 - update parameter values to reduce error.

Image classification using MLP

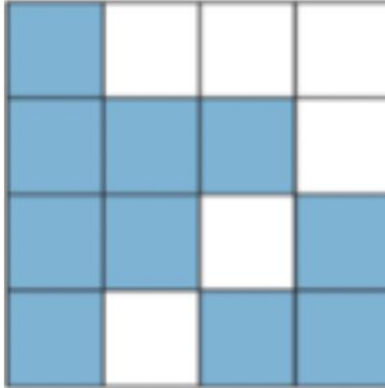


- P
k
- T

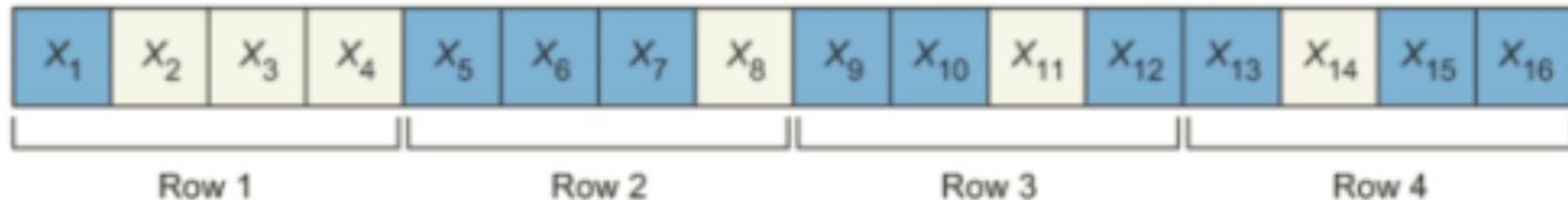


Visualizing Input Vector

- Let's take an example for 4x4 matrix



- Flattened vector is given by



- If pixel value 0 is for black (blue as per figure) and 255 for white, the input vector will look like

$Input = [0, 255, 255, 255, 0, 0, 0, 255, 0, 0, 255, 255, 0, 255, 255, 0, 255, 255]$

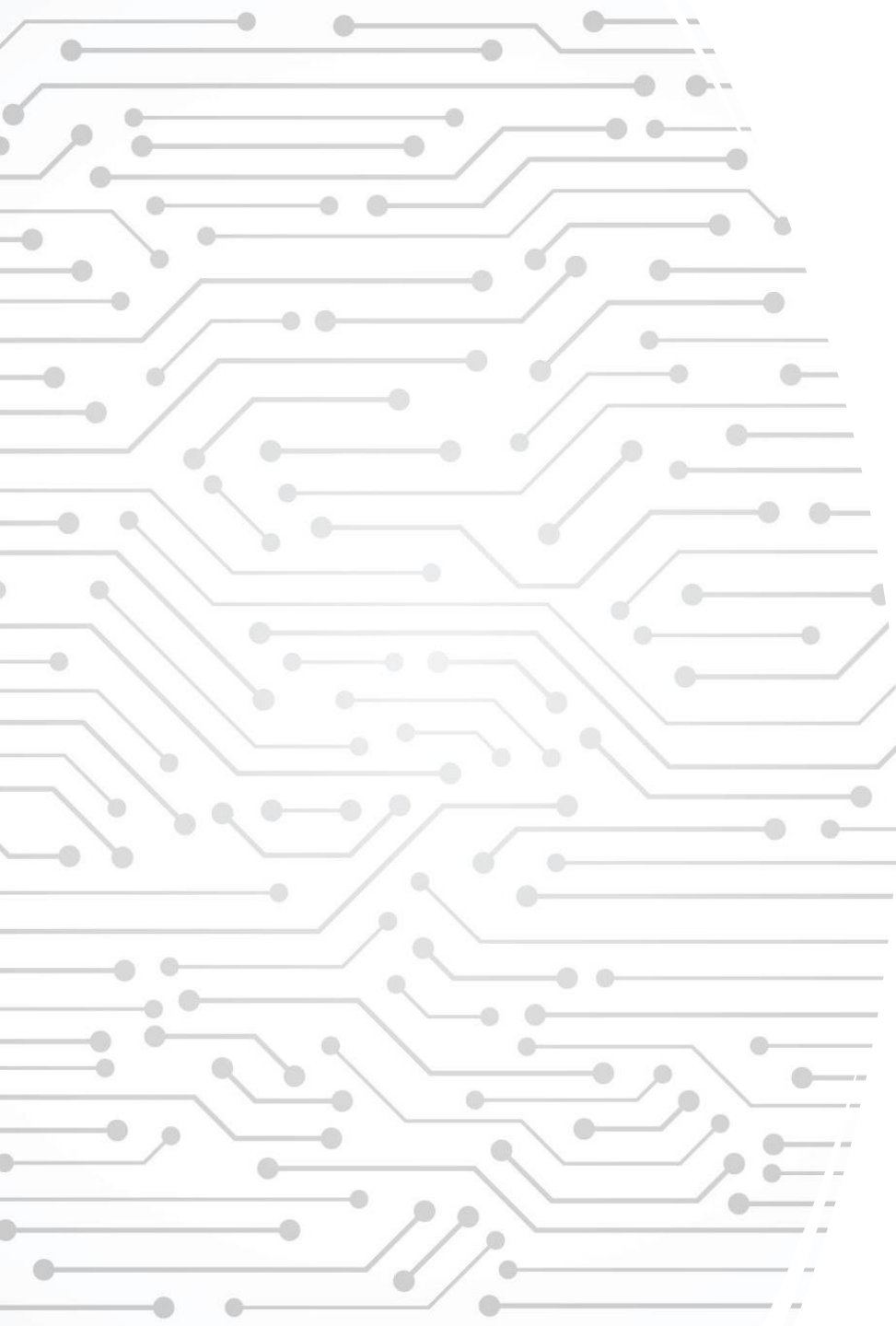
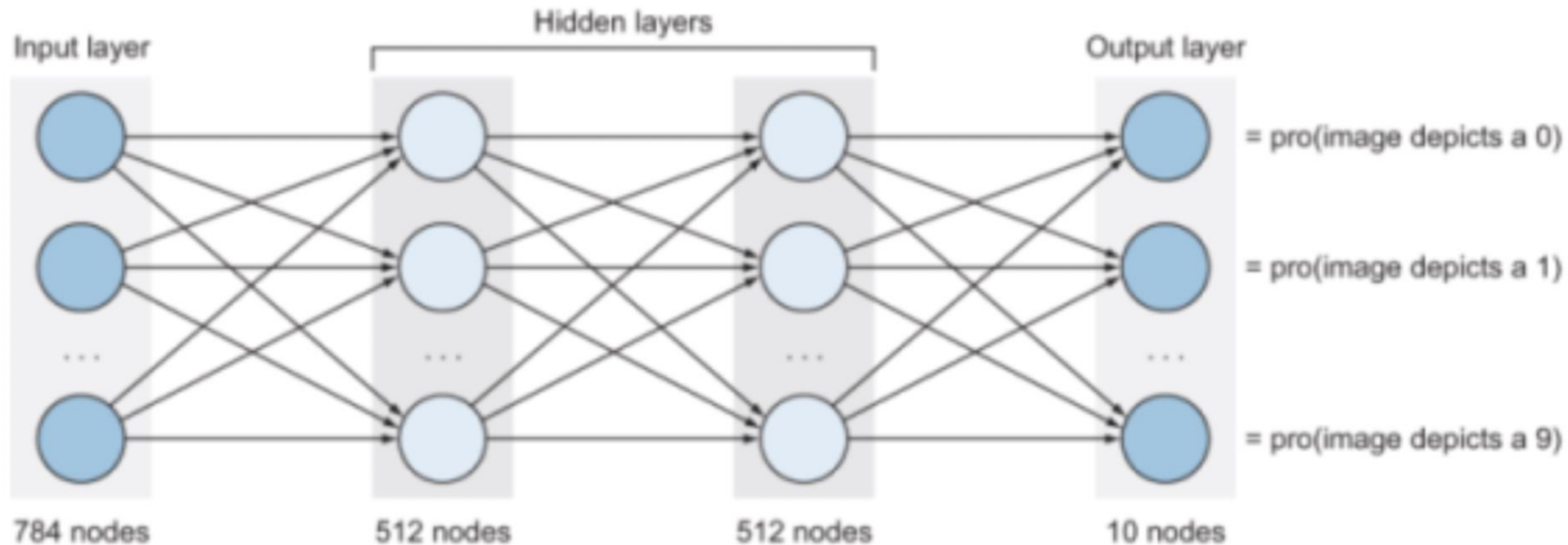


Image processing using MLPs – Hidden Layers

- Any number of hidden layers can be chosen
- For the current example, let's assume 2 hidden layers with 512 nodes each
- Add activation function for each hidden layer
 - ReLU is a very popular activation function for the hidden layers and works well for most of the problems
 - Softmax – used mainly for the output layer when the output classes are mutually exclusive

Image processing using MLPs – Output Layer

- # nodes in output layer = # classes
- Popularly uses Softmax as the activation function

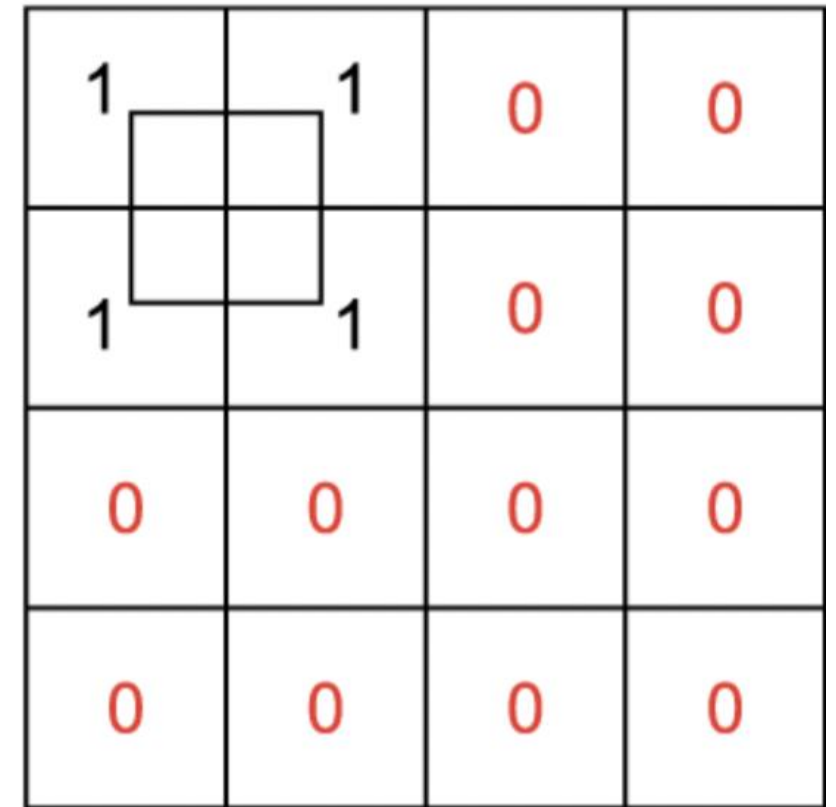


The Model Summary

- Params after the flatten layer = 0
- Params after layer 1 = (784 nodes in input layer) $\times$ (512 in hidden layer 1) + (512 connections to biases) = 401,920.
- Params after layer 2 = (512 nodes in hidden layer 1) $\times$ (512 in hidden layer 2) + (512 connections to biases) = 262,656.
- Params after layer 3 = (512 nodes in hidden layer 2) $\times$ (10 in output layer) + (10 connections to biases) = 5,130.
- Total params in the network = $401,920 + 262,656 + 5,130 = 669,706$.

Drawbacks of MLPs for processing images

- Spatial feature loss
 - MLPs need input to be in 1D vector, (1,n)
 - Flattening the image results in losing the spatial features
 - CNNs accept raw image matrix of pixels
 - Leads to understanding of how the closely located pixels are heavily related to each other as compared those far away
 - E.g. we want to teach the neural network to identify the shape of a square
 - Input vector = [1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]



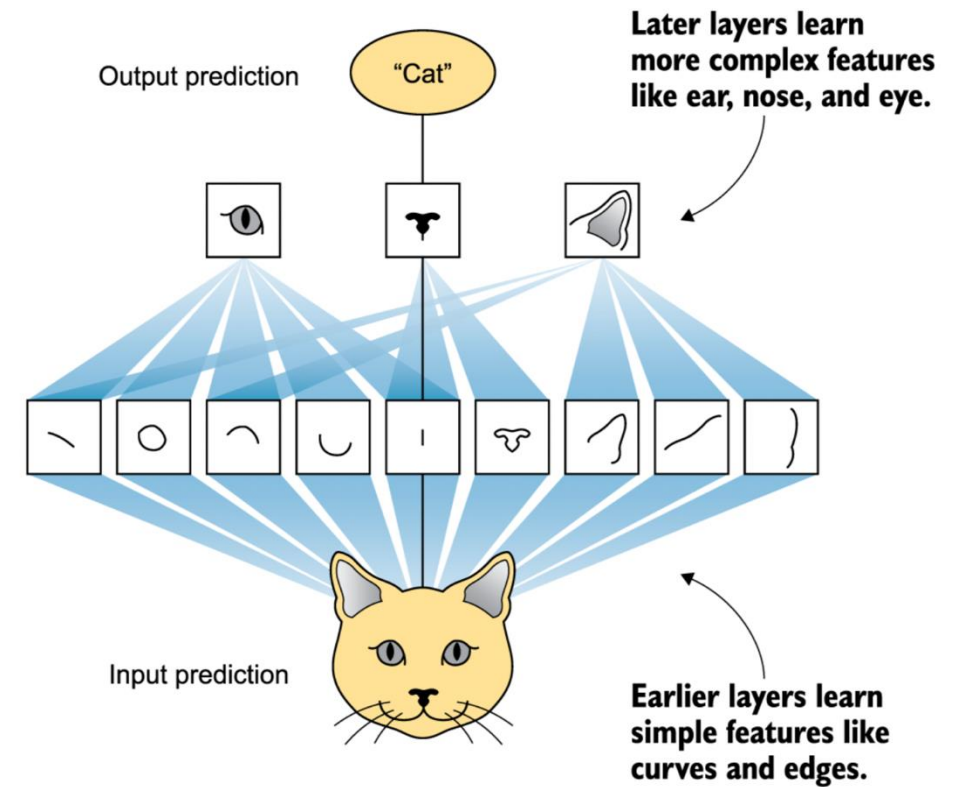
1	1	0	0
1	1	0	0
0	0	0	0
0	0	0	0

Drawbacks of MLPs for processing images

- Square shapes in different areas won't get detected by MLPs
- The network does not learn the square shape as a feature. It just remembered the position of the white pixels
- E.g. images with cats
 - Features to be detected nose, eyes, ears
 - Heavily dependent on the spatial features

0	0	0	0
0	1	0	0
1	1	1	0
0	0	0	0

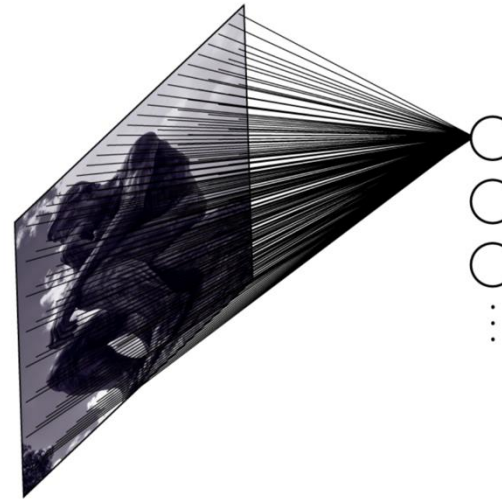
0	0	0	0
0	0	0	0
0	0	1	1
0	0	1	1



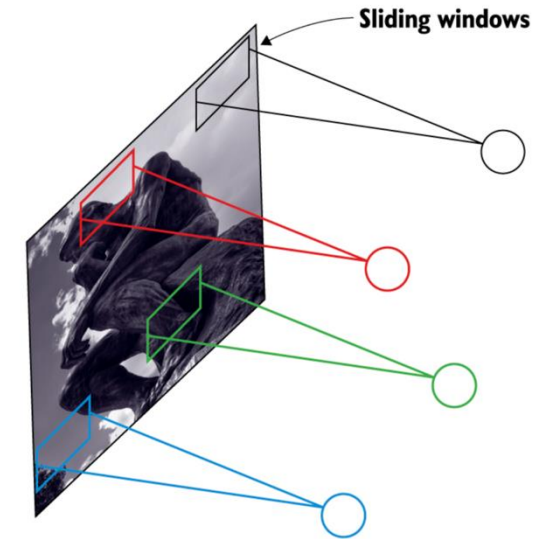
Drawbacks of MLPs for processing images

- Fully connected Layers
 - Works well for the MNIST dataset with image size 28x28
 - For bigger images, e.g. 1000x1000, there will be 1 million parameters for each node in the first hidden layers
 - For 1000 nodes in the hidden layer, the number of trainable parameters only for 1st hidden layer become 1 billion
 - For deep neural networks, it will be very difficult to manage/train the large number of parameters
- CNNs
 - Nodes are connected to only a small subset of the previous layers' nodes
 - Has far fewer trainable parameters than corresponding MLPs
- Summary
 - Need a new method to preserve the spatial information in the image during processing with less compute complexity

Fully connected neural net



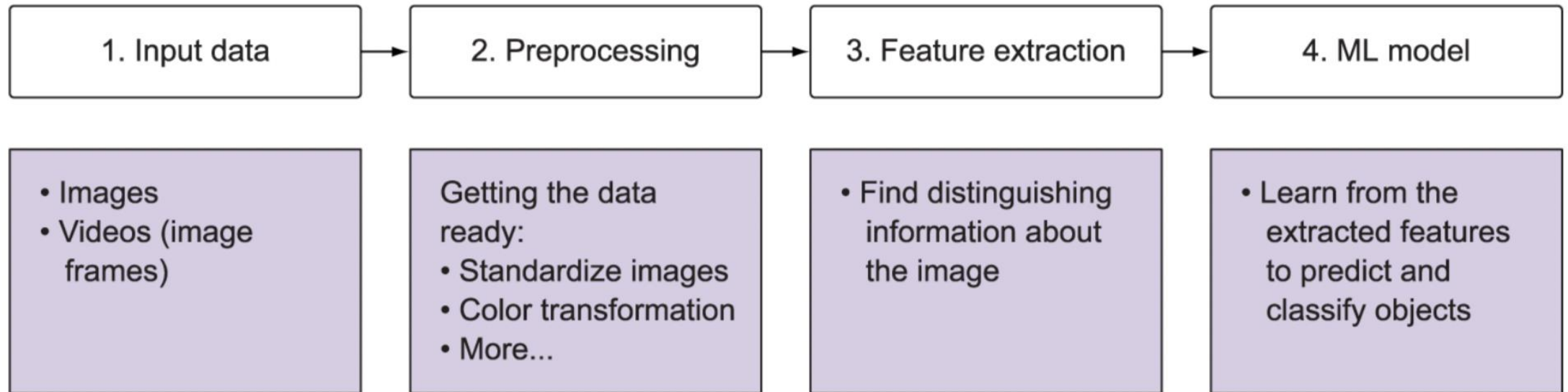
Locally connected neural net



CNN Architecture

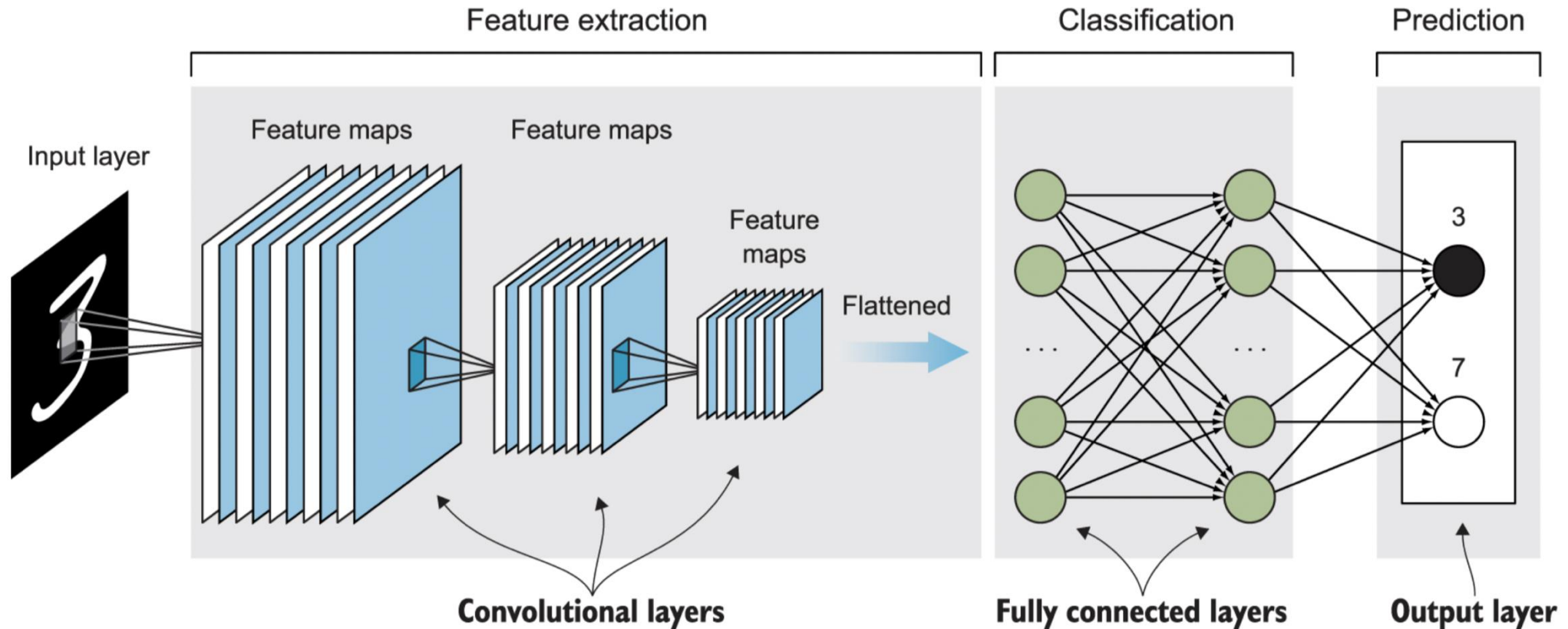
- Initial layer learns edge, lines
- Next layer learns circles, squares by connected features from previous layers
- Following layers detect more complex features such as face, motorbike wheel, ears etc
- Follows similar training process as MLP
 - Input layer, hidden layers, output layer
 - Initializes weights randomly and learned during network training
 - Calculates error
 - Uses gradient descent to update the weights and reduces the cost function

Image Classification Pipeline



- Fully connected layers don't extract features accurately
- However, they are great for classifying extracted features

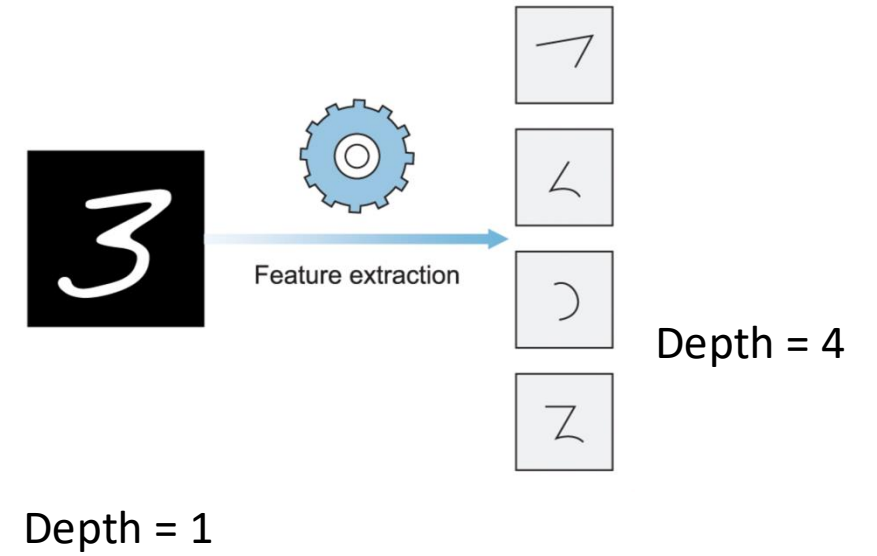
CNN Architecture



- Feature map
 - Mapping of the features in the input image
 - Output of the filter applied to the previous layer

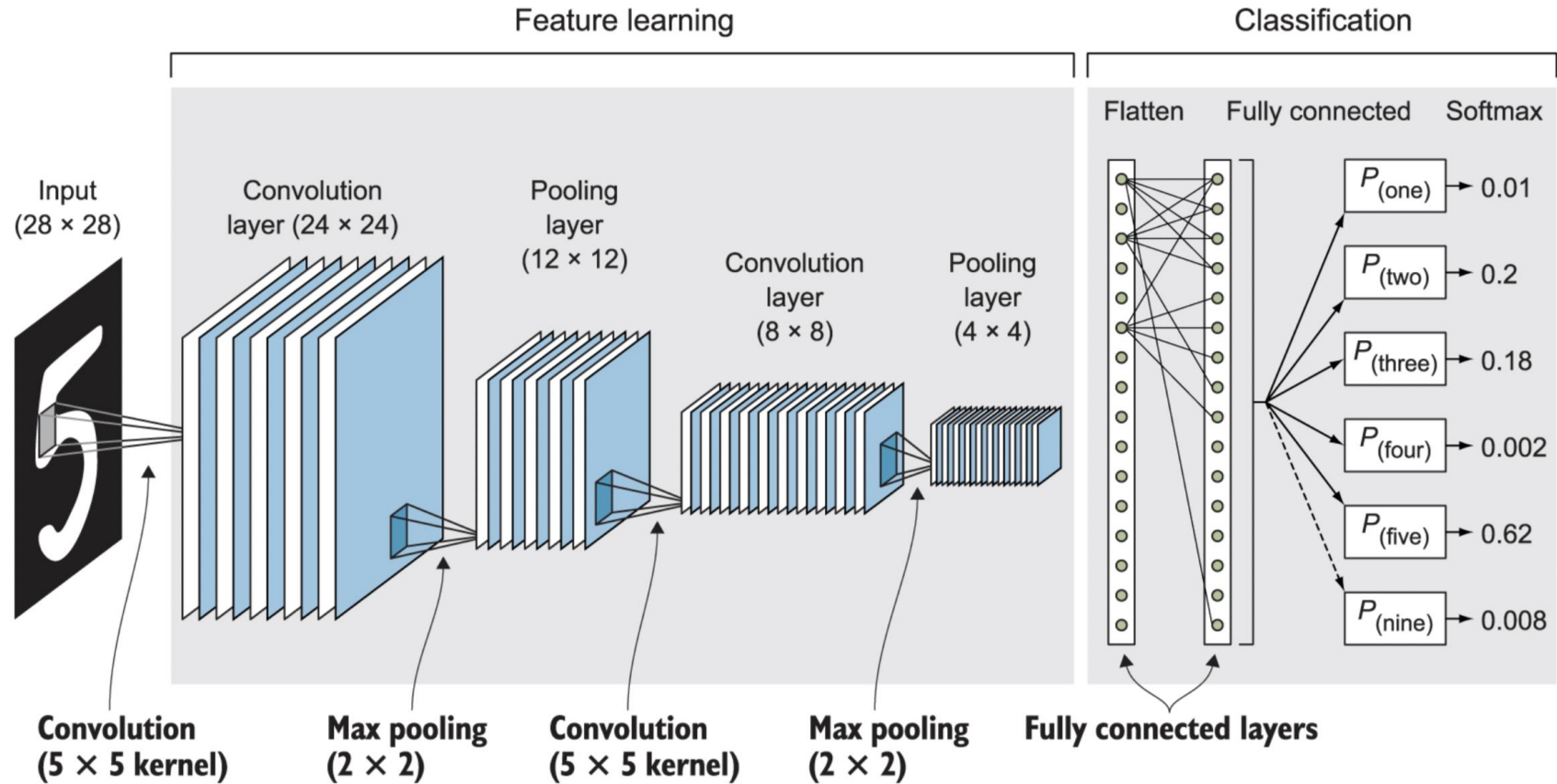
Feature Extraction

- Face detection with CNNs

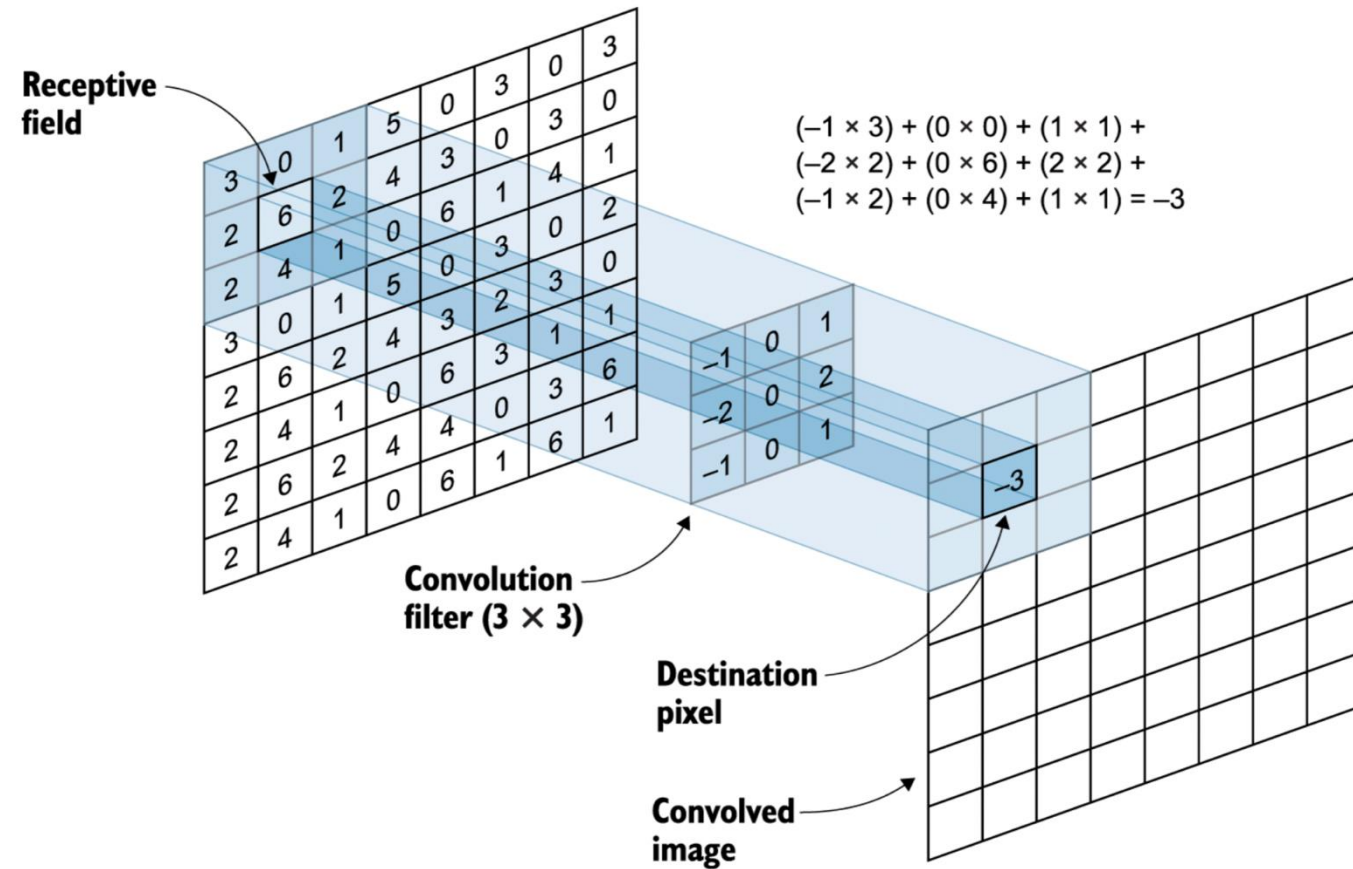


Basic Components of CNN

- Convolutional layer (CONV)
- Pooling layer (POOL)
- Fully connected layer (FC)



Convolutional layer



Convolutional layer

Input image size = $n \times n$

Convolutional filter size = $f \times f$

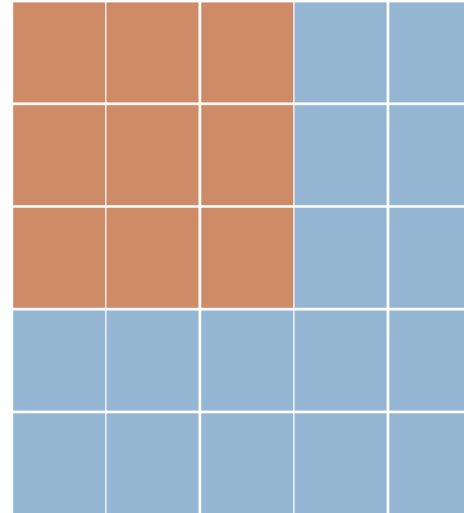
Output size = $(n - f + 1) \times (n - f + 1)$

Input image size = 5×5

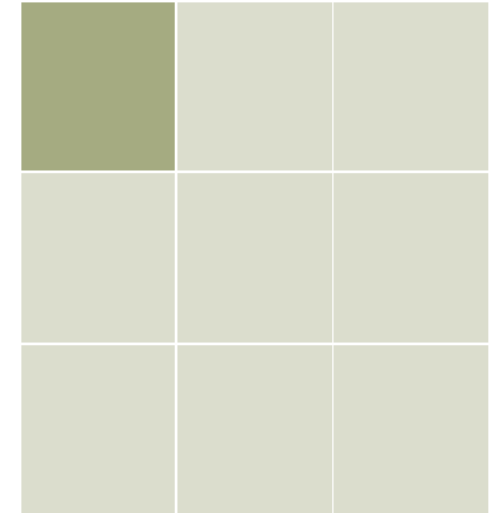
Convolutional filter size = 3×3

Output size = 3×3

Type: conv - Stride: 1 Padding: 0

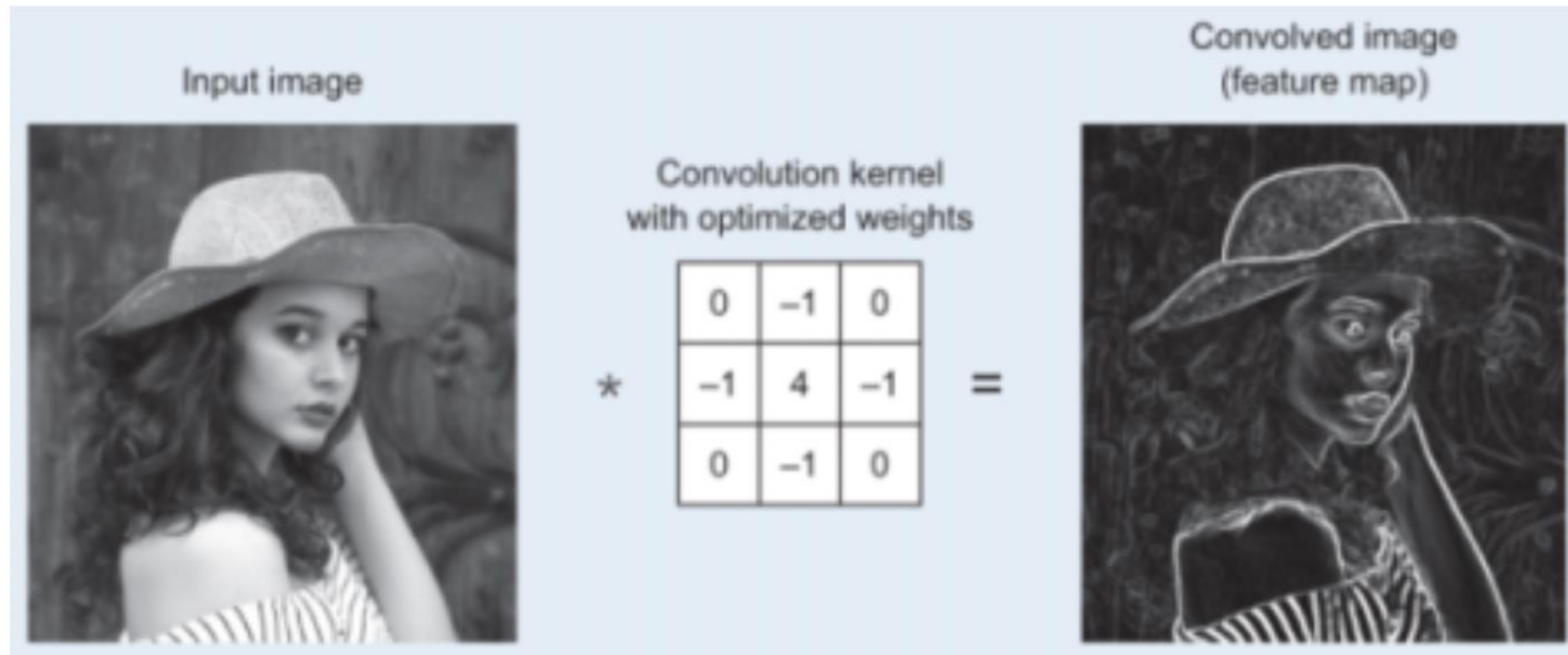


Input



Output

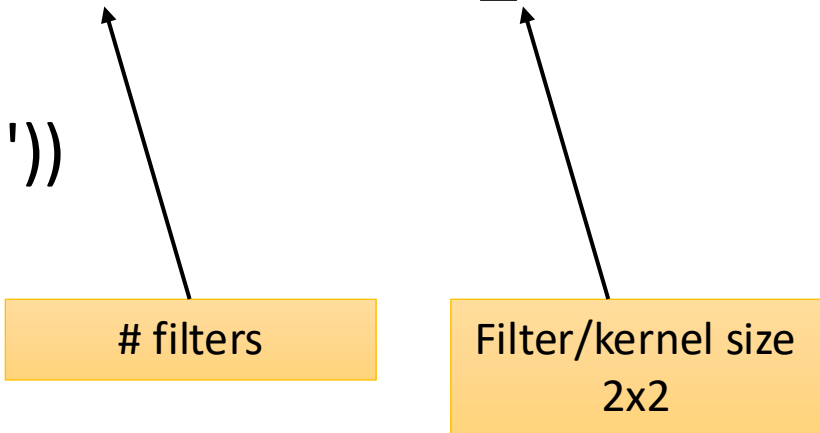
Example after applying a filter to the image



Convolutional layer implementation in Keras

```
from keras.layers import Conv2D
```

```
model.add(Conv2D(filters=16, kernel_size=2, strides='1',  
padding='same',  
activation='relu'))
```

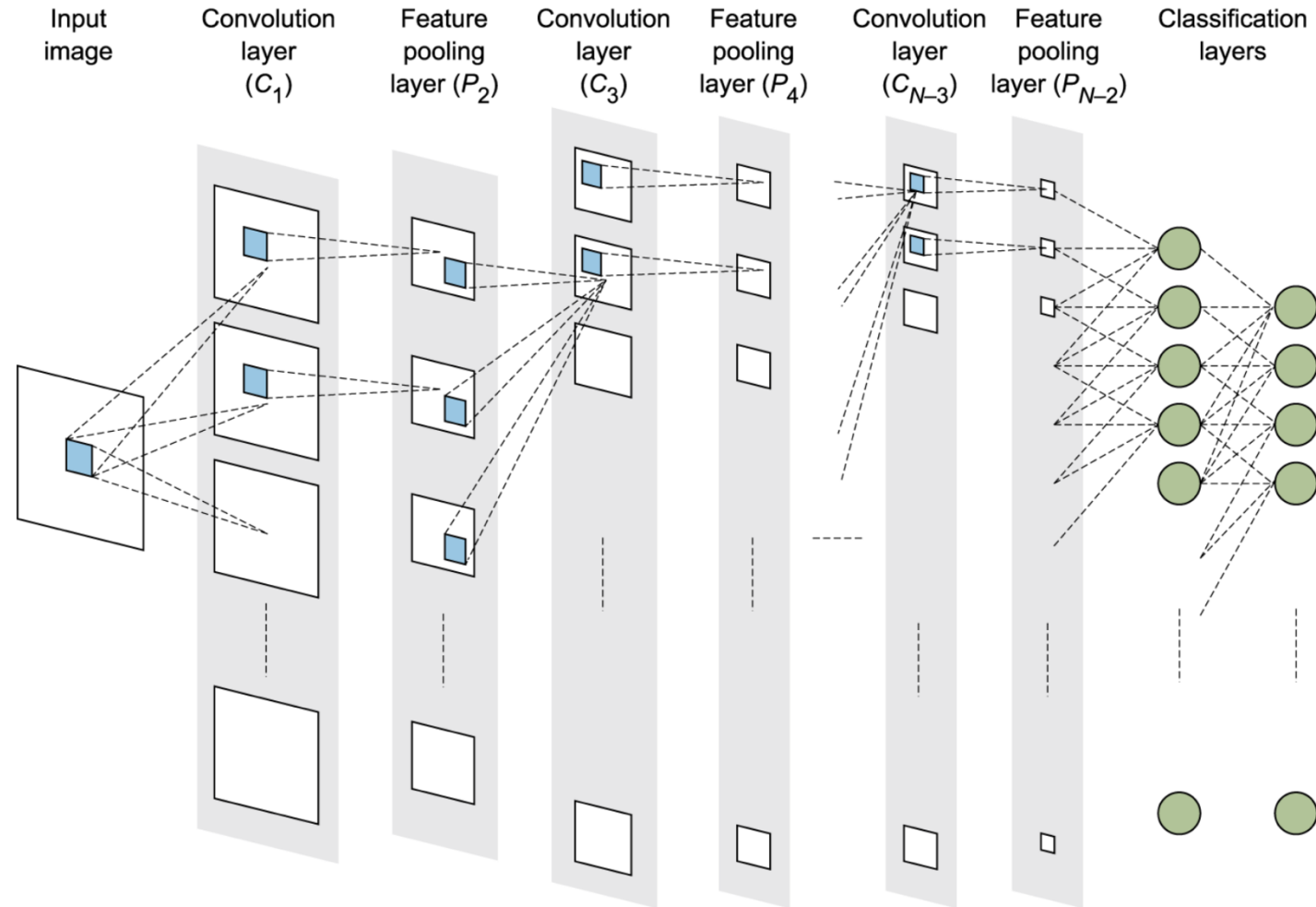


filters

Filter/kernel size
2x2

Number of filters in the Conv layer

- Convolutional layers are the hidden layers
- Each filter can be considered as a neuron



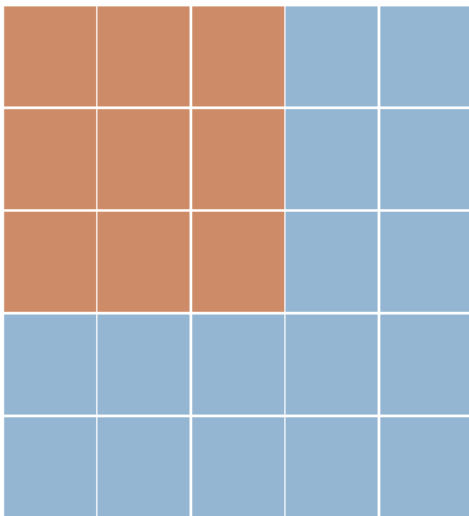
Kernel/filter size

- Always square matrix with size usually 2x2, 3x3 or 5x5
- Small filters can capture fine details of the image
- Bigger filters might lose important information in the image.
- Weights/values in the filter get learned during the model training process

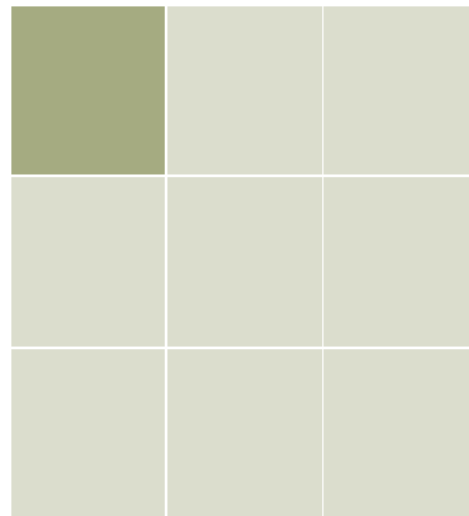
Stride

- The amount by which the filter slides over the input image
- Produces smaller output volumes spatially
- Output matrix size = $\frac{n+2P-f}{s} + 1$

Type: conv - Stride: 1 Padding: 0

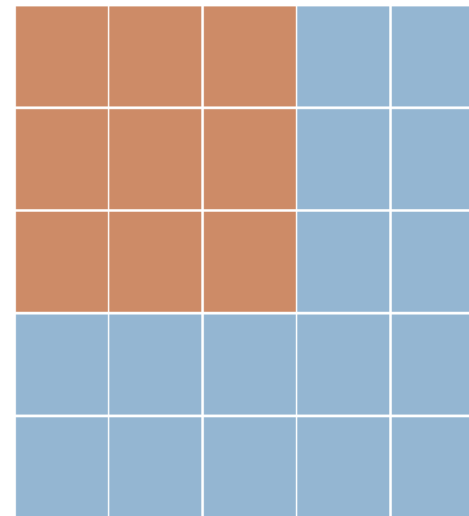


Input

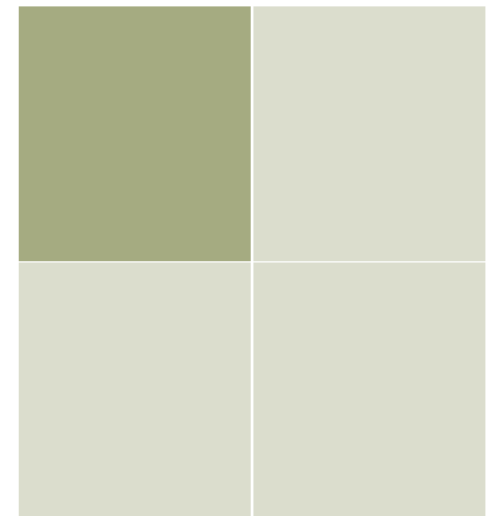


Output

Type: conv - Stride: 2 Padding: 0



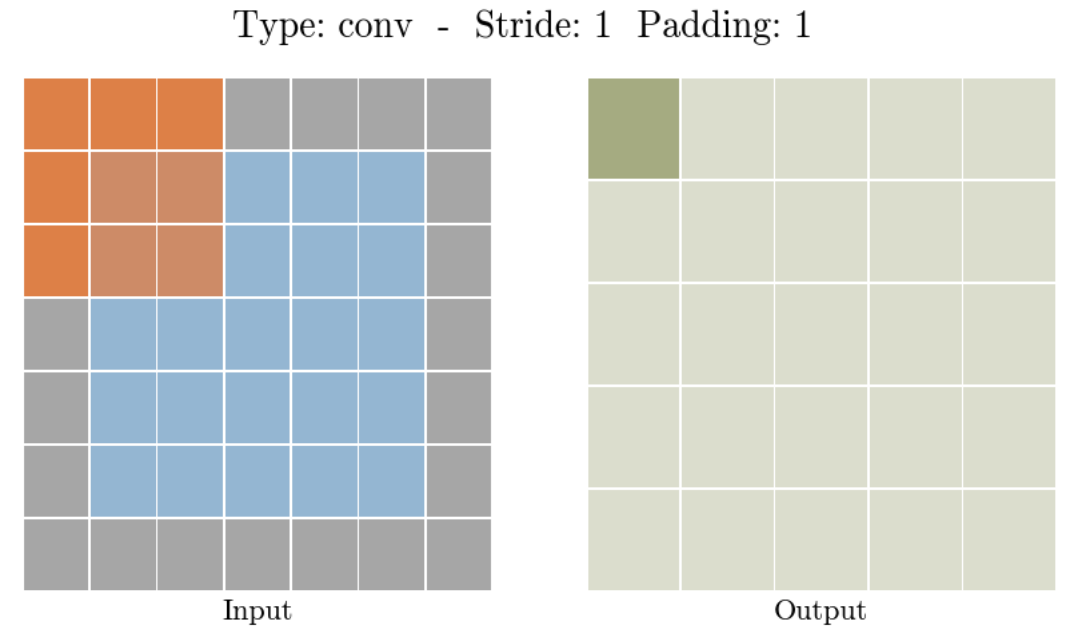
Input



Output

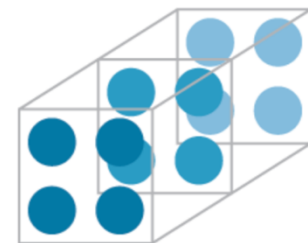
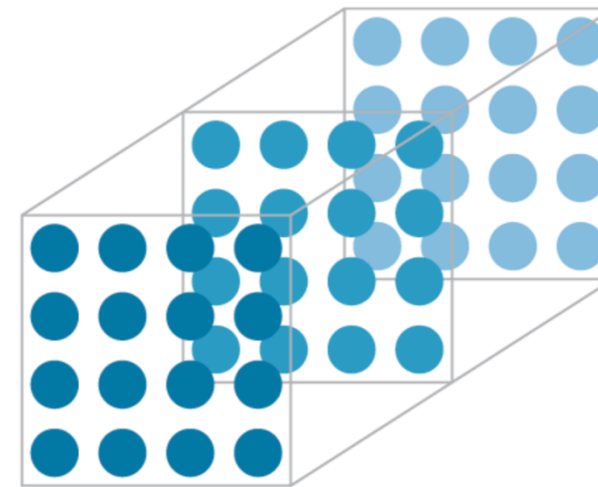
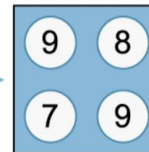
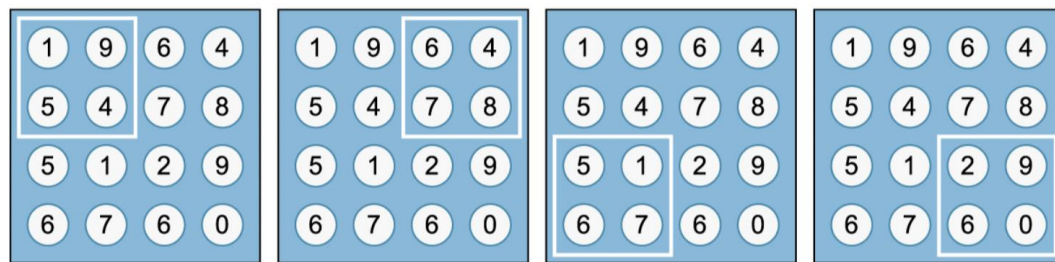
Padding

- Helps with producing output with same size as input
- Output matrix size = $\frac{n+2P-f}{s} + 1$
- When $s=1$ and padding =same,
 - Keeps the important details of the image and pass them to the next layer
- Else
 - Ignore some spatial information to reduce computation complexity

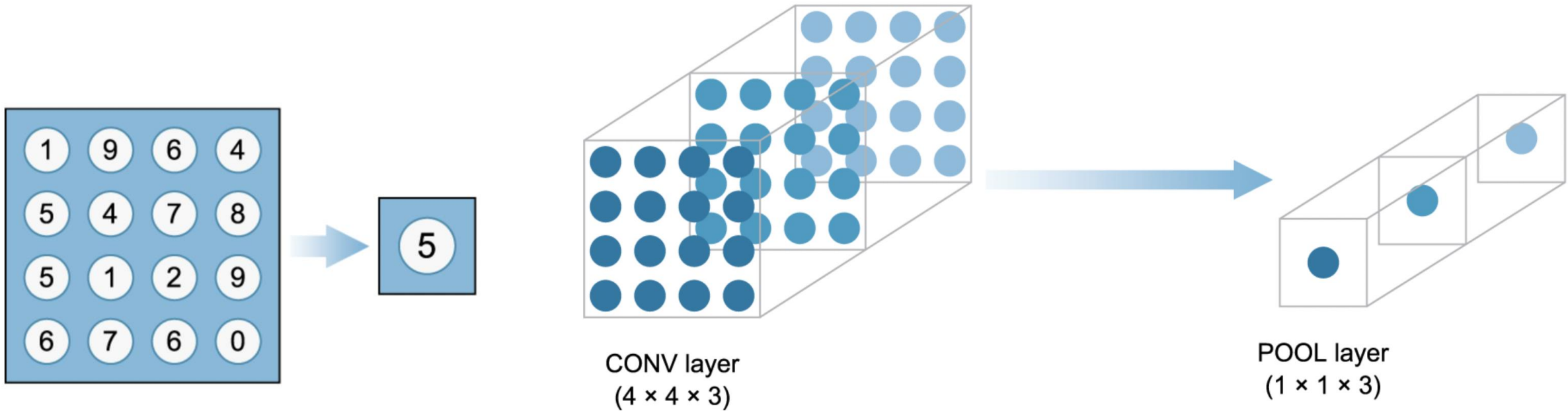


Pooling layer/subsampling

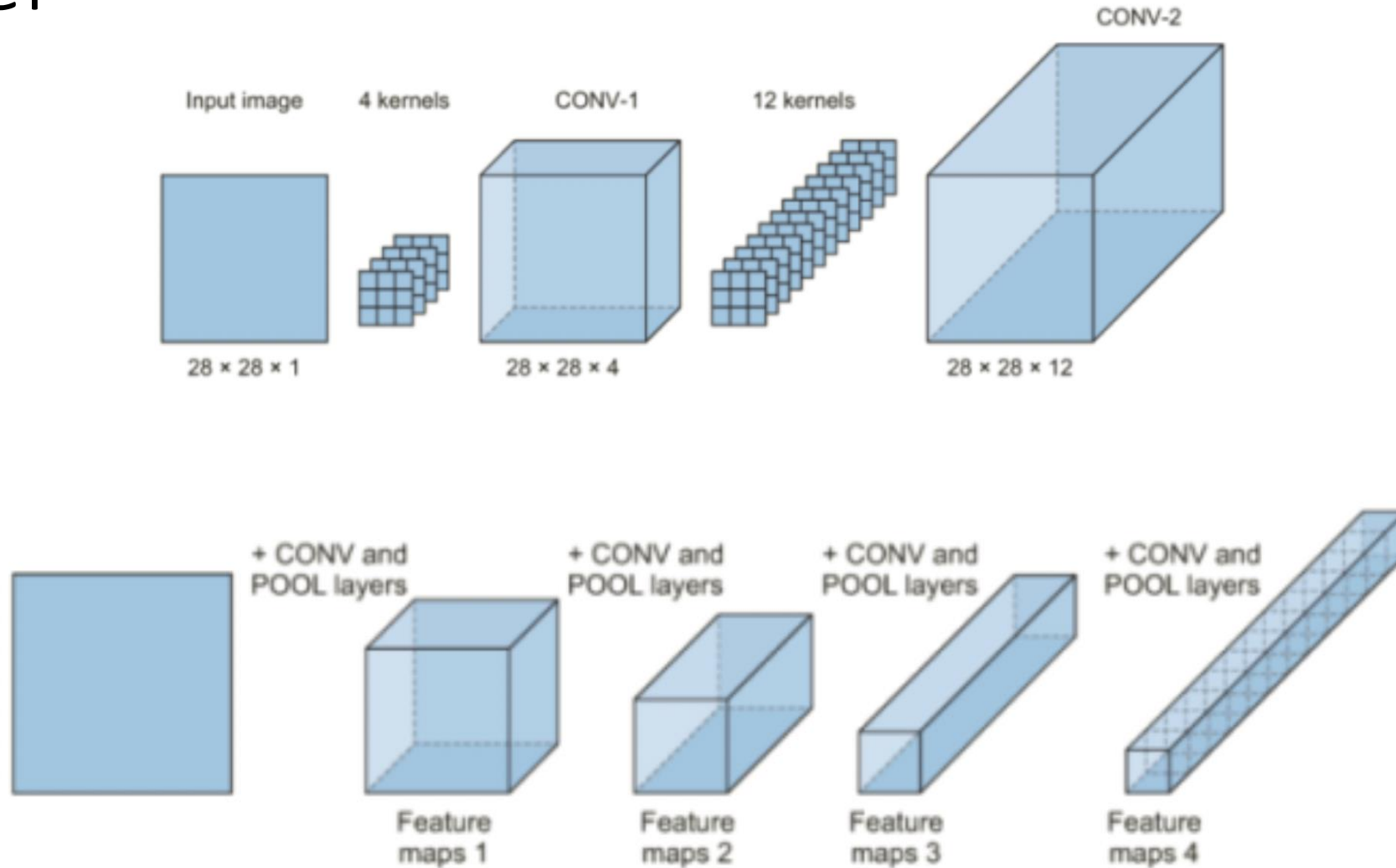
- Down samples the feature map
- Reduces image resolution



Average Pooling

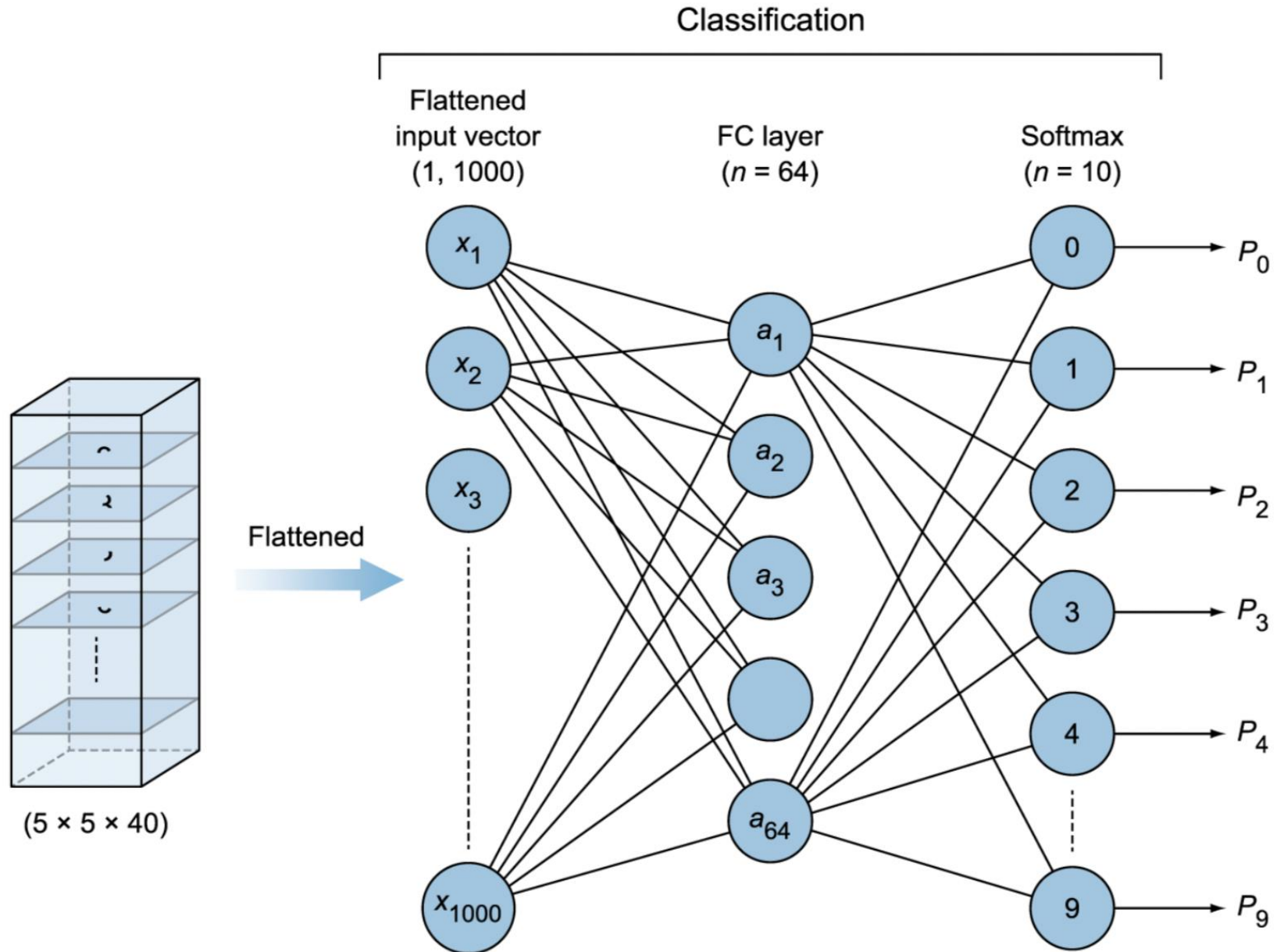


Visualize the network with filters and pooling layer



Fully connected layers

- Classify the input images using extracted features from Conv layers



CNN model building

- `from keras.models import Sequential`
- `from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout`
- `model = Sequential()`
- `model.add(Conv2D(32, kernel_size=(3, 3), strides=1, padding='same',`
• `activation='relu', input_shape=(28,28,1)))`
- `model.add(MaxPooling2D(pool_size=(2, 2)))`
- `model.add(Conv2D(64, (3, 3), strides=1, padding='same', activation='relu'))`
- `model.add(MaxPooling2D(pool_size=(2, 2)))`
- `model.add(Flatten())`
- `model.add(Dense(64, activation='relu'))`
- `model.add(Dense(10, activation='softmax'))`
- `model.summary()`

CNN model summary

- # trainable parameters = filters x kernel size x depth of the previous layer + bias (# of filters)

```
model.add(Conv2D(32, kernel_size=(3, 3),  
strides=1, padding='same',  
activation='relu', input_shape=(28,28,1)))
```

#trainable parameters = $32 * 3 * 3 * 1 + 32 = 320$

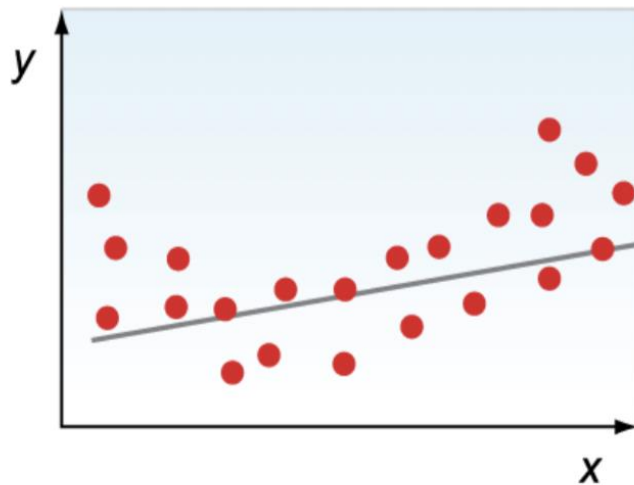
```
model.add(Conv2D(64, (3, 3), strides=1,  
padding='same', activation='relu'))
```

Trainable parameters = $64 * 3 * 3 * 32 + 64 = 18496$

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d_1 (MaxPooling2)	(None, 14, 14, 32)	0
conv2d_2 (Conv2D)	(None, 14, 14, 64)	18496
max_pooling2d_2 (MaxPooling2)	(None, 7, 7, 64)	0
flatten_1 (Flatten)	(None, 3136)	0
dense_1 (Dense)	(None, 64)	200768
dense_2 (Dense)	(None, 10)	650
Total params: 220,234		
Trainable params: 220,234		
Non-trainable params: 0		

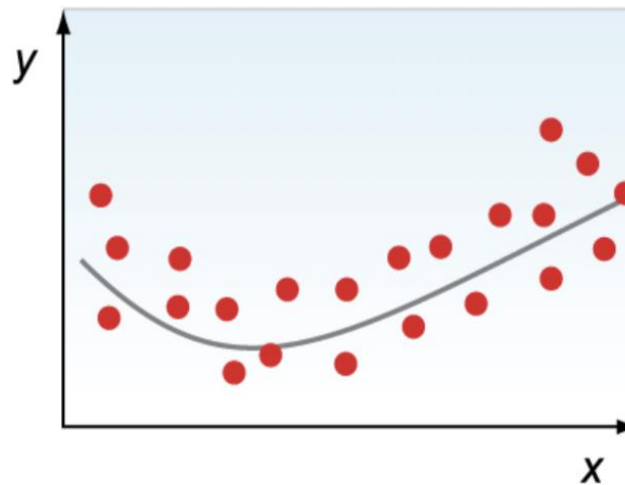
Model training

Underfitting



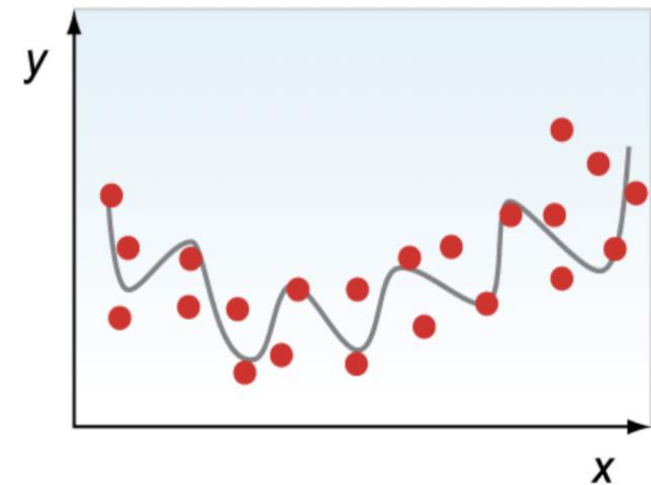
- Fails to fit the training data
- Simple model

Just right!



- Fits training data well
- Predicts well on on unseen data

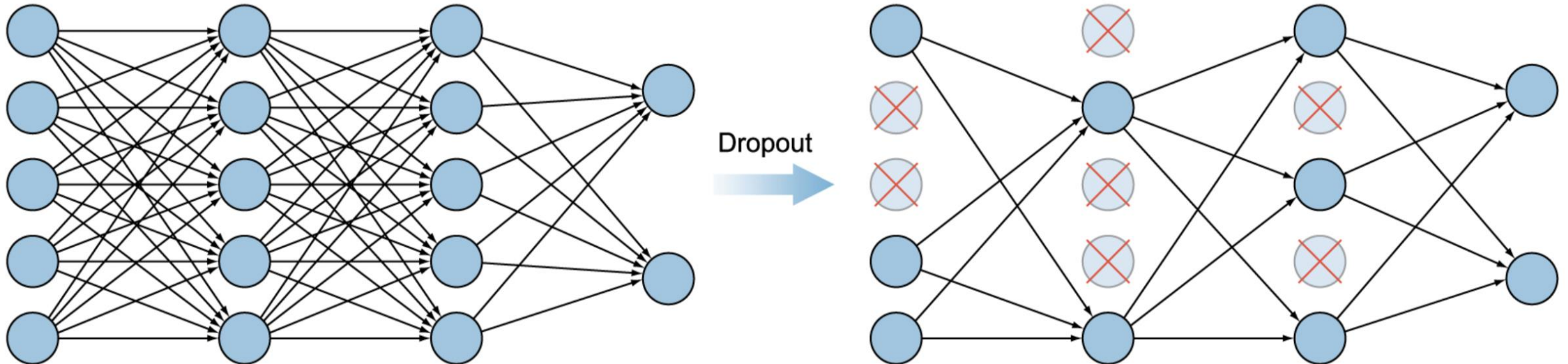
Overfitting



- Fitting the data too much
- Memorizing training data, no feature learning

Dropout layer

- Used to prevent overfitting
- Hyperparameter that needs to be tuned during training
- Randomly selects nodes to be removed from every epoch during training



Dropout layer

- Enforces each node to learn the patterns in the image without relaying on a few features
- Reduces interdependent learning among neurons
- A form of ensemble learning
- Gets added in fully connected layers after the image flattening
 - Keras code
 - `model.add(Dropout(rate=0.3))`
 - 30% of the neurons will automatically get dropped every epoch

Convolutional layer

Input image size = $n \times n$

Convolutional filter size = $f \times f$

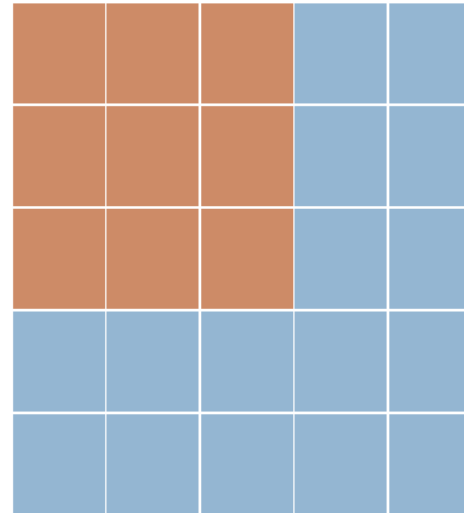
Output size = $(n - f + 1) \times (n - f + 1)$

Input image size = 5×5

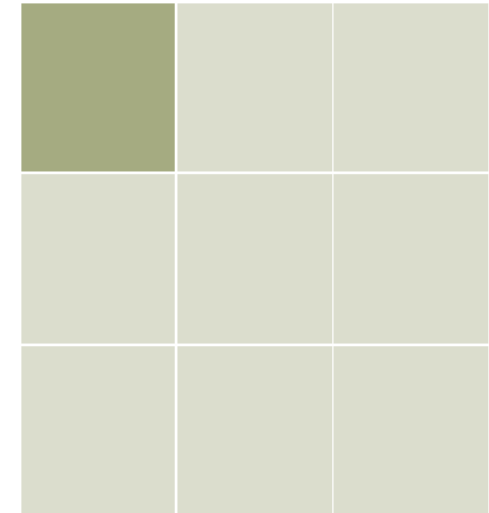
Convolutional filter size = 3×3

Output size = 3×3

Type: conv - Stride: 1 Padding: 0



Input



Output

Color images

Color image

$F(0, 0) = [11, 102, 35]$



RGB channels

Channel 3
Blue intensity
values

35	165	163	165	158	...
166	166	164	166	159	...
156	158	162	165	159	...
...	...	...	...	...	...

Channel 2
Green intensity
values

102	169	167	169	169	...
170	170	168	170	170	...
160	162	166	169	170	...
...	...	...	...	...	...

Channel 1
Red intensity
values

11	158	156	158	158	...
159	159	157	159	159	...
149	151	155	158	159	...
146	146	149	153	158	...
145	143	143	148	158	...
...	...	...	...	...	...

Filter/Kernel

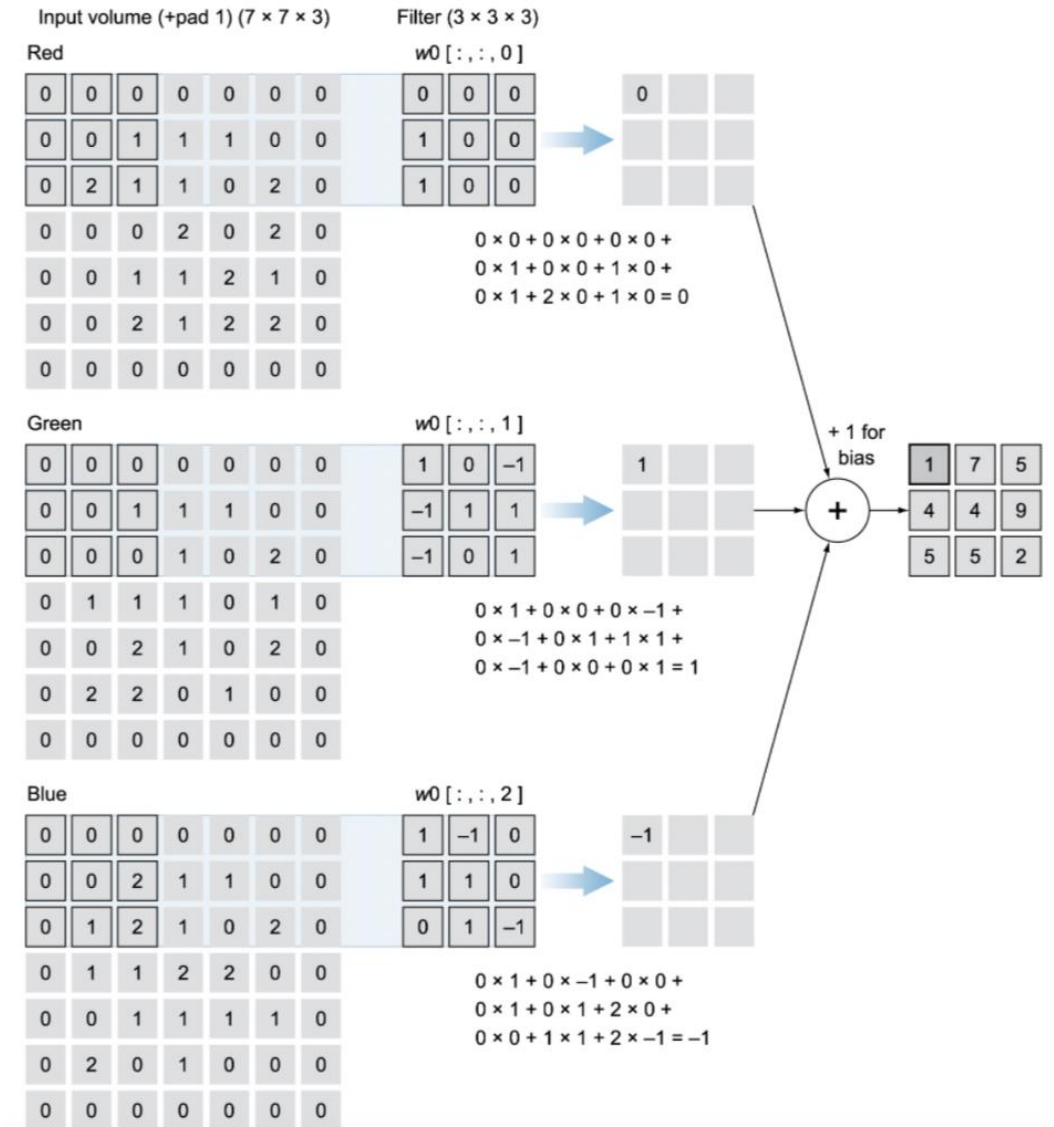
-1	-1	+1
-1	+1	-1
+1	-1	-1

-1	-1	+1
-1	+1	-1
+1	-1	-1

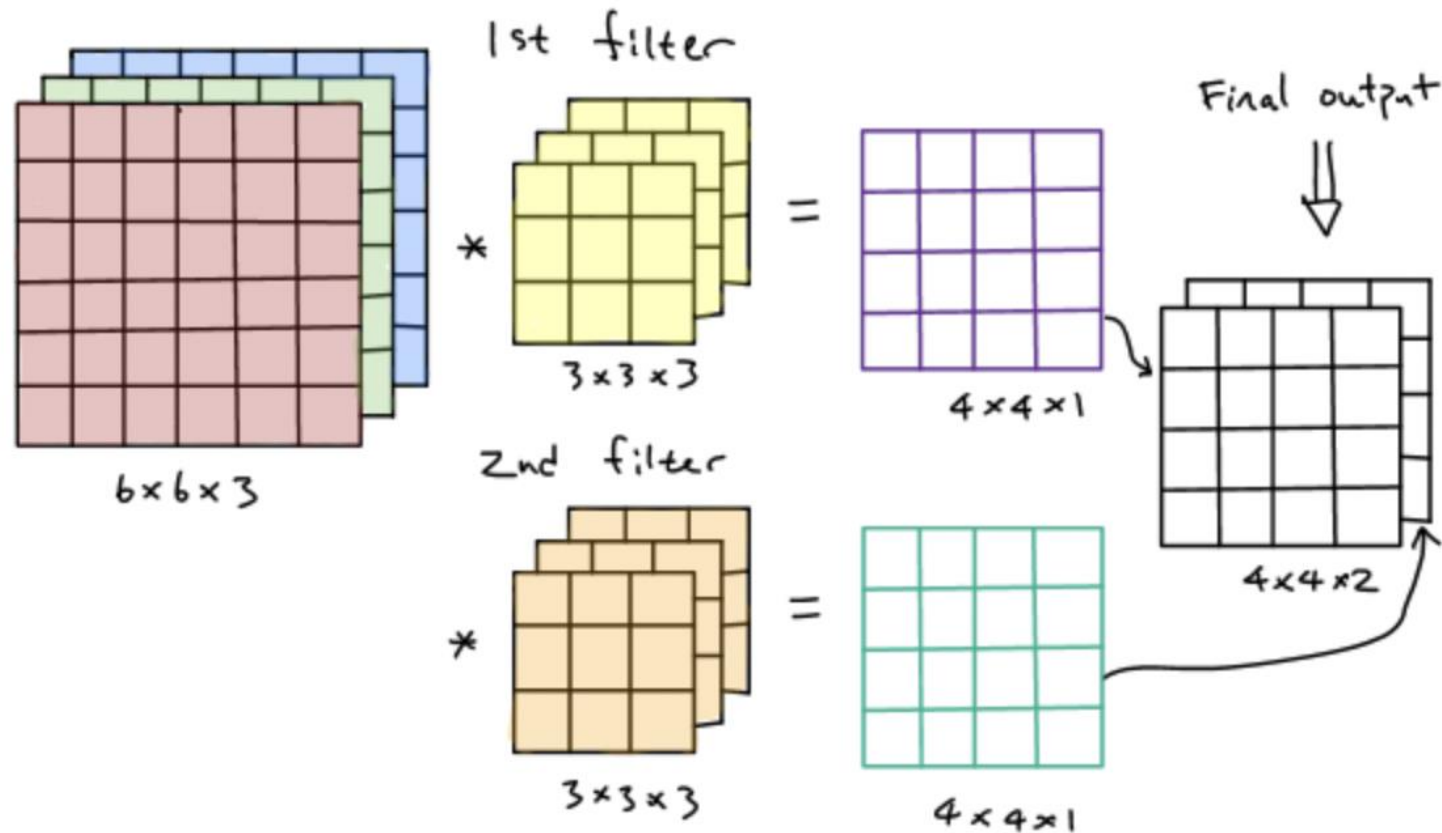
-1	-1	+1
-1	+1	-1
+1	-1	-1

Convolutions on color images

- Increase in number of trainable parameters
- Filter corresponding to each channel
- Slide the filters over every channel to do element-wise multiplication and add them to obtain convoluted value of the pixel in the respective channel
- Add all the the convolved pixel values from every channel along with bias to obtain single node value



Convolutions of a color image with multiple filters



Computational Complexity for color images

- Number of parameters for a 3x3 filter
 - Grayscale image – 9
 - Color image – 27
- Requires more memory
- Increased computation complexity
- Color images contain more information as compared to grayscale images
 - Color vs gray scale images - application dependent

Reference

- Book – Deep Learning for Vision Systems by Mohamad Elgendy, Manning Publication, November 2020